# Network-Aware Predictive Digital Twin Framework for Resilient and Self-Reconfigurable Cyber-Physical Production Systems

**Abstract.** Delayed observations and missing sensor transitions complicate supervisory decisions in conveyor-based production cells. This paper presents a simulation-stage digital twin framework that separates process simulation, sensing, communication, estimation, sequence control, prediction, and independent evaluation. Software reconfiguration is restricted to a cycle-latched speed limit and a hold request; it neither changes the mechanical layout nor automatically resumes an interrupted workpiece. A command-history estimator propagates workpiece position between timestamped optical observations. A reduced predictor evaluates 54 stopping trajectories over a 0.5-second horizon, while an independent MuJoCo belt-and-workpiece predictor provides a more detailed alternative. A matched study comprises 24 simulated runs covering nominal operation, missing pickup detection, communication delay, packet loss, and command-to-motion mismatch. Eighteen runs complete placement and six terminate in a supported hold, without violating the stated terminal criteria. In the two delayed-observation cases, enabling reconfiguration permits completion whereas disabling it results in a hold. A separate Docker timing exercise finds that the reduced predictor completes all 15 measured calls within a configured 20-millisecond allowance. The MuJoCo predictor completes none within that allowance and remains unsuitable for that deadline in its current implementation. These observations support traceable software-level experimentation, rather than measured fault-diagnosis accuracy or verified physical resilience. Native grouping and configurable workers reduce complete-forecast time in checked cases but do not meet the 20-millisecond target. Physical calibration, live controller integration, and hardware validation remain unreported.

**Keywords:** Digital twin · Network-aware supervision · State estimation · Software reconfiguration · Virtual commissioning

## 1 Introduction

A geometric representation alone does not establish that a digital twin can support production decisions. Kritzinger et al. [7] distinguish digital models, shadows, and twins according to data integration. For a moving workpiece, a decision additionally depends on when an observation was generated and what motion

occurred before its reception. Communication delay and packet loss are established concerns in networked control [5]; the age of timestamped information provides a representation of freshness [12]. Here, digital twin framework denotes the intended architecture. Its reported implementation is a simulation testbed, not an already synchronized physical twin.

### 1.1 Related Research and Remaining Development Needs

Negri et al. [8] reviewed the roles of digital twins in production systems and emphasized sensor-enabled synchronization of virtual representations with physical processes. Uhlemann et al. [11] proposed multimodal data acquisition and guidelines for constructing production-system twins. These studies establish the importance of an acquired and updated process-data basis. For this cell, that requirement motivates explicit source timestamps and observation boundaries rather than assuming that a geometric model supplies the current process state.

Jones et al. [6] characterized digital twins through entities, states, connections, and twinning rate, and identified model fidelity and technical implementation among further research needs. Fuller et al. [4] reviewed enabling technologies and open challenges across digital-twin applications. These reviews provide context for separating representation, data exchange, and useful services. They do not validate a particular timing allowance or guarantee that increasing model detail improves an online supervisory decision.

Cimino et al. [2] examined manufacturing applications, their services, and integration with production control, and presented a laboratory implementation involving a manufacturing execution system. Rasheed et al. [9] reviewed digital twins from a modeling perspective, including physics-based and data-driven approaches. Together, these perspectives motivate evaluating both the decision service and its computational model. Here, the narrower development need is to record whether a forecast is complete, timely, and based on admissible observations before using it in the specified continue-or-hold service; this is not a claim that earlier studies omitted all such concerns.

Ademujimi and Prabhu [1] combined equipment and discrete-event simulation to generate interventions for learning Bayesian-network structures for manufacturing fault diagnostics. This provides a precedent for exploiting virtual faults when physical fault data are limited. The present objective is different: an unresolved observation must lead to a continue-or-hold decision while a workpiece is moving. A unique physical fault cause cannot simply be assumed from one missing binary transition.

Fu et al. [3] developed a digital-twin-based framework that monitors disturbances, optimizes manufacturing-line configurations, and evaluates alternatives through generated simulation. Their work addresses line-level configuration and disturbance handling. We instead fix the equipment arrangement and investigate software actions: limiting conveyor speed or requesting a hold. The resulting evaluation must distinguish continued production from interruption and must not count a supported hold as a completed product.

MuJoCo supplies multibody and contact dynamics for model-based control [10]. Engine selection alone does not establish calibrated pneumatics or guarantee that an online forecast finishes before the next decision. We therefore separate plant simulation from two supervisory prediction alternatives and measure computation time explicitly. The focused research question is whether timestamped observations and bounded motion forecasts support traceable decisions under controlled disturbances without exposing simulator ground truth to the supervisor. This synthesis of selected relevant studies is not a systematic review or evidence that the complete literature lacks such integration.

### 1.2 Scope and Contributions

The implemented contributions are an observation-constrained modular architecture; a command-history estimator and continue-or-hold supervisor; a cycle-latched speed-degradation policy; and reproducible virtual experiments with separate timing measurements. Resilience is examined through terminal support and completion in specified fixtures, not through a general resilience index. Self-reconfiguration refers to the software speed limit and hold, not mechanical rearrangement, sensor replacement, or autonomous recovery. A later exploratory component learns optical-event arrival residuals and supplies an advisory event monitor; a separate segment-level comparison evaluates speed selection. Physical-validation sections remain intentionally unfilled.

## 2 System and Information Boundaries

### 2.1 Cell and Hybrid Representation

The cell comprises two conveyors, a transfer pusher, three photoelectric-sensor roles, an XY gantry, a vertical actuator, and a vacuum gripper. It handles one workpiece per reset. Nominal workpiece dimensions are $60 \times 60 \times 60$ mm in the parameterized profile; these are preliminary user-reported dimensions, not a physical measurement campaign reported here. CAD V5 and the functional MuJoCo scene are not claimed to be fully synchronized. Mass, friction, suction-related quantities, and motion response remain engineering assumptions awaiting identification.

The hybrid architecture can be written as

$$x_{k+1} = F_\theta(x_k, u_k, q_k), \qquad q_{k+1} = G(q_k, z_{\le k}, \widehat{x}_k), \tag{1}$$

where $x_k$ denotes continuous simulated states, $q_k$ the sequence phase, $u_k$ actuator requests, and $\theta$ the parameter profile. Equation (1) describes the software architecture, not a newly identified physical state equation. The sequence covers admission, pushing, transport, alignment, pickup, transfer, release, return, completion, and latched hold. Endpoint switches support cylinder transitions. Independently observed feedback for XY completion, intermediate vertical positioning, and vacuum confirmation is not fully available in the evaluated configuration.

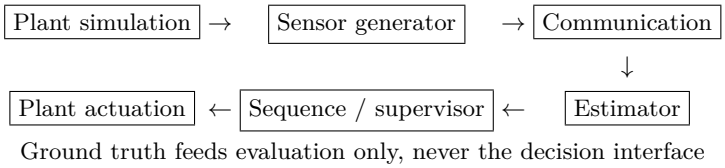



**Fig. 1.** Virtual information paths. Communication delivers observations to the estimator; true state is excluded from the decision interface.

### 2.2 Observation-Constrained Decisions

Figure 1 separates the decision path from evaluation. Simulated state supplies the sensor generator and an independent evaluator. The estimator receives only observations delivered through the communication model. Missing feedback remains unavailable rather than silently becoming a negative sensor reading. Messages carry session identity, reset generation, and sequence number; wrong-session, old-generation, and repeated messages are rejected. These local checks are not an authenticated industrial communication protocol.

## 3 Estimation and Predictive Supervision

### 3.1 Information Age and Motion Estimation

Let an accepted observation have source time $\tau_i$ and reception time $r_i$. At decision time $t$, its information age is

$$a(t) = t - \tau_{\ell(t)}, \qquad r_{\ell(t)} \le t, \tag{2}$$

where $\ell(t)$ indexes the newest accepted observation. Age concerns generation time, not simply time since reception [12]. Evaluation uses a shared simulation clock. Real PLC clock offset and synchronization uncertainty have not been measured.

Between optical edges, the estimator integrates issued belt commands instead of reading simulator velocity. Let $D_i = \int_{\tau_{i-1}}^{\tau_i} u_b(t)\,dt$, $V_i$ be command-speed total variation, and $N_i$ count commanded stops. The implemented propagation is

$$\widehat{y_i^-} = \widehat{y}_{i-1} + \widehat{g} D_i + N_i c_s, \tag{3}$$

$$b_i^- = b_{i-1} + \rho(\tau_i - \tau_{i-1}) + \tau_u V_i + N_i c_s. \tag{4}$$

Here $\widehat{g}$ is an estimated command-to-motion gain, $c_s$ a stopping-coast allowance, $\rho$ drift allowance, and $\tau_u$ command-response uncertainty. The interval half-width $b_i$ is an engineering allowance, not a calibrated confidence interval.

An optical rising edge refers to a leading face, not the center. With beam coordinate $p_j$, workpiece length $L$, and sample-window correction $h_i = \widehat{g} D_i/2$, the history-enabled estimator resets position and allowance as

$$\widehat{y}_i = p_j - L/2 + h_i, \quad b_i = b_0 + \max\{h_i, g_{\max} D_i + N_i c_s - h_i\}. \tag{5}$$

Gain is updated from commanded travel between belt-sensor edges, subject to plausibility checks. Commanded motion after the observation source time is accounted for separately when deciding whether pickup uncertainty is acceptable. Admission centering still depends on provisional geometry and commanded feed motion; arbitrary arrivals are outside the evaluated scope.

### 3.2 Finite-Scenario Continue-or-Hold Decision

The reduced predictor uses estimated position $\widehat{y}$, velocity $\widehat{v}$, uncertainty $b$, and observation age $a$. For uncertainty offset $\epsilon$, velocity offset $\nu$, reaction delay $\delta$, and action $\ell \in \{0, 1\}$, define $v_\nu = \max(0, \widehat{v} + \nu)$ and $b_a = b + \rho a$. The stopping surrogate is

$$\begin{aligned} y_{\ell,\epsilon,\nu,\delta}(s) = \widehat{y} + \epsilon b_a + v_\nu a + v_\nu \min(s, \delta + \ell T) \\ + c_s \operatorname{clip}\left(\frac{s - \delta - \ell T}{h}, 0, 1\right), \end{aligned} \tag{6}$$

where $T$ is the control interval and $h$ the reduced prediction grid. In history-enabled operation, the supplied forecast speed is an upper bound derived from issued speed and allowed gain, rather than an independently measured belt velocity. Action $\ell = 0$ requests hold now; $\ell = 1$ allows one additional interval before stopping. The coast term is not identified braking physics. For forward limit $y_{\text{lim}} = y_{\text{end}} - L/2 - d_{\text{reserve}}$, the action margin is

$$m_\ell = \min_{\epsilon,\nu,\delta,j} \{y_{\text{lim}} - y_{\ell,\epsilon,\nu,\delta}(jh)\}. \tag{7}$$

Continuation requires a complete forecast, $m_1 \geq 0$, and completion within the computation allowance. Otherwise the supervisor requests hold. If $m_0 < 0$, holding is not thereby proved safe; the rule still rejects additional continuation. This is a supervisory screen, not an optimal-control or stability theorem.

### 3.3 Contact Forecast and Software Reconfiguration

The alternative predictor constructs an independent MuJoCo belt/workpiece scene from estimates and the profile, without cloning live plant state. It evaluates belt continuation and stopping and checks forward, lateral, and vertical margins. It excludes the full gantry, pressure dynamics, and calibrated suction failure. Its fixed-footprint checks are not a general rotated-body collision certificate. The two predictors thus have different model detail and numerical workloads.

Repeated age warnings or observation gaps can latch NORMAL into DEGRADED. In transport, the fixed policy issues

$$u_b^{\text{issued}} = \min\{u_b^{\text{requested}}, u_{\text{cap}}\}. \tag{8}$$

Unusable observations, contradictory endpoint switches, missing expected transitions, or supervisory violations can request HOLD. Belt requests then become

zero while the existing vacuum request is retained; this does not confirm physical vacuum. Communication recovery does not clear the hold automatically. The implemented reset handshake prepares a fresh simulation cycle through distinct reset, acknowledgement, and start steps. It does not resume a previous suspended workpiece.

## 4 Virtual Evaluation Methods

### 4.1 Settings and Timing Measurements

The profile uses a 2 ms physics step and a 20 ms control interval. Both predictors evaluate 54 trajectories: two actions, three position offsets, three velocity offsets, and three reaction delays. The horizon is 0.5 s. The reduced grid is 20 ms, while contact integration uses 2 ms; equal scenario counts do not imply equal work. Nominal and degraded transport commands are 0.16 and 0.12 m/s. They are selected simulation settings, not measured physical capacities.

Three synthetic estimated states are used for timing. Each receives one warm-up call and five measured calls per predictor and allowance, yielding 15 measured calls per group. Initialization is recorded separately and warm-up samples are retained outside the summaries. External elapsed time includes the invocation wrapper and result creation. We report medians, nearest-rank empirical p95, completion counts, and observed deadline exceedances. The configured allowance is 20 ms; a separate 1 s functional allowance permits inspection of complete forecasts without replacing the configured deadline.

The recorded Docker environment contains Python 3.12.16, MuJoCo 3.16.0, and NumPy 2.5.4 on WSL2 Linux. Processor identity and controlled background load were not recorded for this campaign. Timing is therefore descriptive, not a worst-case execution-time bound or a cross-platform benchmark.

### 4.2 Matched Disturbances and Evaluation Criteria

Six scenarios vary one factor: nominal operation; pickup-sensor stuck low; 20 ms message delay; 5% packet loss; and plant belt speed at 0.85 or 1.15 times its command. True plant-speed changes are not simultaneously supplied to the estimator as known gains. Each scenario uses two network seeds and two policy conditions, giving 24 runs. The conditions differ in whether the fixed-policy reconfiguration module, comprising the cycle-latched cap and health-triggered holds, is enabled. This is not an isolated speed-cap ablation. Both retain estimation and predictive supervision: this is not a sequential-PLC-versus-DT comparison. Changing seeds does not make deterministic scenarios independent replicates.

Full-cycle trials explicitly enable open-loop assumptions for unconfirmed XY, intermediate vertical positioning, and vacuum feedback. Strict operation instead holds before these steps. Results under these assumptions do not demonstrate physical feedback control. A terminal criterion is met by evaluator-confirmed placement after completion or evaluator-confirmed support during hold. Speed-cap adherence and policy logging are also checked. Ground truth is restricted to

**Table 1.** Observed Docker forecast timing; 15 measured calls per row after warm-up

| Model | Allowance (ms) | Median (ms) | p95 (ms) | Complete /15 | Overruns /15 |
|---|---|---|---|---|---|
| Reduced 1D | 20 | 2.76 | 3.54 | 15 | 0 |
| Reduced 1D | 1000 | 2.30 | 3.02 | 15 | 0 |
| MuJoCo contact | 20 | 20.48 | 21.09 | 0 | 15 |
| MuJoCo contact | 1000 | 460.37 | 596.29 | 15 | 0 |

evaluation. This study uses the 1 s functional prediction allowance, so terminal outcomes do not establish 20 ms operation. No threshold tuning is performed within the campaign.

## 5 Virtual Results

### 5.1 Forecast Computation

Table 1 is generated from the recorded 2 November Docker evidence, before the implementation optimization reported below. The reduced predictor completes all 15 measured calls under the configured allowance, with p95 3.54 ms. The contact predictor completes none of its 15 calls within that allowance. With the functional allowance, it completes all 15 with p95 596.29 ms. Its incomplete 20 ms calls request hold; returning hold after an observed overrun is not a hard-deadline guarantee. The current contact predictor is not demonstrated to support a 20 ms supervisor on this host. The result does not show that the reduced model is physically more accurate.

### 5.2 Disturbance Outcomes

All 24 runs meet the terminal, cap, and logging checks: 18 complete placement and 6 end in supported holds. Table 2 preserves this distinction. In the two delayed-observation cases, enabling reconfiguration permits completion; disabling it produces holds for excessive pickup uncertainty. Both conditions hold when pickup detection is absent. These are fixture-level observations, not a diagnosis sensitivity, false-alarm rate, or throughput gain.

The earlier artifact also records 97 logic checks and end-to-end integration checks for strict/open-loop operation, replay, prediction, and deadline handling. Check counts support implementation consistency; they are not performance sample sizes. Source hashes and configurations are retained with the experiment records. No physical result is inferred from passing these checks.

### 5.3 Native Rollout Optimization

A subsequent implementation replaces repeated Python stepping with the installed native rollout facility. It groups candidates by their stopping-step index,

**Table 2.** Matched reconfiguration-module study: completion (D) and supported hold (H)

| Condition | Off: D | Off: H | On: D | On: H |
|---|---|---|---|---|
| Nominal | 2 | 0 | 2 | 0 |
| Pickup sensor stuck low | 0 | 2 | 0 | 2 |
| Delay: 20 ms | 0 | 2 | 2 | 0 |
| Loss: 5% | 2 | 0 | 2 | 0 |
| Plant speed: 0.85 command | 2 | 0 | 2 | 0 |
| Plant speed: 1.15 command | 2 | 0 | 2 | 0 |

**Table 3.** Native rollout observations on 3 November: full-call median and p95 use the 1 s allowance; each count is out of 15 measured calls

| Implementation | Median (ms) | p95 (ms) | Complete at 20 ms | Complete at 1 s |
|---|---|---|---|---|
| Reference loop | 398.45 | 447.00 | 0 | 15 |
| Native: 1 worker | 137.37 | 151.62 | 0 | 15 |
| Native: 4 workers | 56.39 | 62.52 | 0 | 15 |
| Native: 8 workers | 50.29 | 54.65 | 0 | 15 |

evaluates running prefixes, and starts the stopped-model segment with the same time, position, and velocity transfer as the reference implementation. Every returned step, including the initial state, contributes to the margin check. Physics parameters, the integration grid, and the candidate set are retained. Worker count is a separate software setting; configurations with one, four, and eight workers are evaluated rather than silently selected as a new default.

Twelve numerical fixtures, including zero speed, aged estimates, end-of-belt conditions, and off-grid stopping times, are checked at each worker count against the retained reference. Actions, scenario counts, integration-step counts, and minimum margins satisfy the declared checks; the observed maximum margin difference is zero with a comparison tolerance of $10^{-9}$ m. This tolerance tests two implementations of the same model and is not a statement about physical accuracy. A separate trajectory regression compares individual native states with successive reference steps.

Table 3 reports a separate 3 November Docker timing campaign, with 15 measured calls per implementation and allowance after warm-up. The observations show lower complete-forecast times for native grouping and additional workers. Calls are evaluated in sequence on the same host, without randomized ordering or controlled background load; no universal speedup or statistical significance is claimed. No variant completes a measured call within 20 ms. Native groups cannot be interrupted mid-call, and returning HOLD after a detected overrun can take longer than in the reference loop. The optional eight-worker end-to-end simulation also holds for a prediction-deadline overrun with evaluator-confirmed support. It is not counted as a completed product. Thus the optimization im-

**Table 4.** Paced observation-only Docker cycles on 3 November; foreground runtime-call timing, not end-to-end latency

| Condition | Steps | Median (ms) | p95 (ms) | Maximum (ms) | Over 20 ms (steps) |
|---|---|---|---|---|---|
| Disabled | 697 | 0.32 | 2.59 | 4.92 | 0 |
| Observation only | 697 | 0.47 | 3.51 | 12.63 | 0 |
| Incomplete worker | 697 | 0.45 | 3.01 | 4.57 | 0 |

Observation-only cycle: 35 requests, 34 admitted observations, 1 discarded.
Incomplete-worker cycle: 81 incomplete forecasts rejected.

proves the observed full-forecast computation but does not establish a real-time physical supervisor.

### 5.4 Observation-Only Background Forecasts

To examine separation of computation from supervision, a spawned process runs the contact predictor while the reduced predictor retains command authority. A bounded channel permits one outstanding request and skips new submissions while busy. No controller, actuator writer, or live plant state is supplied to the worker. Each request binds a session, generation, profile digest, source sequence and time, estimate snapshot, and command revision. Returned results are screened for matching provenance, age, operating state, and complete finite scenario output. Simulation-time age and parent-process elapsed age are evaluated separately; this is not physical clock synchronization.

These checks admit observations only. In particular, a continuation forecast computed for an earlier one-interval action does not authorize continuation from the present state. Accepted and rejected replies are logged after the foreground command gate and cannot change a command or clear a hold. Worker initialization and shutdown occur outside the decision step; a response timeout disables observation processing without restarting the worker within that step.

Three paced virtual cycles compare background disabled, background enabled, and an intentionally incomplete background worker. The foreground allowance remains 20 ms. Each uses the same plant and open-loop feedback assumptions. Logged simulation times, phases, estimates, and commands are compared exactly, excluding session identifiers and measured computation durations. A separate probe terminates the worker and checks that the foreground call still returns. This probe tests containment of that software failure, not continued physical production after arbitrary failures.

Table 4 describes the recorded Docker run. All three cycles finish placement as confirmed by the evaluator and have identical compared traces. Complete background observations are received, incomplete results are discarded, and worker processes are closed. The probe that terminates the worker also passes. Timing covers the foreground runtime call including channel polling, but excludes

plant integration, file output, pacing, initialization, and shutdown. Each condition contains one cycle; repeated decision steps are not independent production trials. These measurements do not qualify complete system deadlines or show that the contact forecast itself finishes within 20 ms.

### 5.5 Offline Model-Decision Audit

An offline diagnostic evaluates the reduced predictor from the recorded inputs sent to the contact predictor. Both use the same estimate, source time, decision time, and profile. Its computation clock is disabled only for this offline replay; its equations and scenario set remain unchanged. The replayed action is therefore a model diagnostic, not the actual foreground decision under a deadline. Session, profile, timestamp, sequence, scenario-count, and finite-output checks precede comparison. Incomplete contact forecasts are excluded, and complete replies discarded at reception remain a separate group. Forward-only reduced margins and contact margins with lateral and vertical constraints are not treated as measurements of the same physical error.

In a separate Docker run, the observer cycle yields 24 complete pairs: 22 admitted observations and two discarded replies. Model actions agree for all 24 snapshots in that fixture. The deliberately incomplete worker yields 82 replies and zero comparable pairs. Three cycles retain identical compared control traces and meet the placement criterion. The incomplete-worker cycle also records one of 697 foreground calls over 20 ms, with a maximum of 32.75 ms. This observation is retained and precludes claiming uniformly observed 20 ms steps for that run. These are correlated observations from single virtual cycles, not independent production trials or estimates of physical fault-detection accuracy. The audit does not grant command authority to either a stale reply or a model-agreement count.

### 5.6 Exploratory Boundary and Step-Size Sensitivity

To move beyond nominal recorded snapshots, a fixed exploratory grid combines eight distances to the permitted center limit ($-5$, 0, 5, 10, 15, 20, 40, and 80 mm), three estimated speeds (0.08, 0.16, and 0.24 m/s), two observation ages (0 and 60 ms), and three sliding-friction multipliers (0.25, 1, and 2). The resulting 144 input states each use an assumed 2 mm position allowance. Sliding friction is scaled for both belt and workpiece; other friction components are retained. These are synthetic sensitivity inputs, not identified physical distributions or complete production cycles. A 5 s functional computation allowance permits inspection of complete model output without qualifying the 20 ms control interval.

Each input is evaluated by the reduced predictor and two contact variants. The contact variants retain identical dynamics but score either the forward margin alone or the original combined forward, lateral, and vertical margin. All 144 inputs complete in the recorded Docker run. Forward-only and combined contact actions coincide in this grid, and the combined margin never exceeds its forward-only counterpart beyond the $10^{-9}$ m comparison tolerance. This is a

**Table 5.** Fixed exploratory boundary grid: disagreement of reduced and forward-contact actions; complete inputs per row, not production trials

| Friction scale | Age (ms) | Complete | Reduced C / contact H | Reduced H / contact C |
|---|---|---|---|---|
| 0.25 | 0 | 24/24 | 2 | 0 |
| 0.25 | 60 | 24/24 | 0 | 1 |
| 1 | 0 | 24/24 | 0 | 0 |
| 1 | 60 | 24/24 | 0 | 1 |
| 2 | 0 | 24/24 | 1 | 0 |
| 2 | 60 | 24/24 | 0 | 1 |

C: continue; H: hold. Full-contact and forward-only actions coincide on this grid.

constraint-ablation check, not evidence that lateral or vertical constraints can generally be removed.

The reduced and forward-contact actions agree on 138 inputs. Three inputs produce reduced continuation with contact hold, and three produce the reverse. Thus disagreement is present even with a shared forward-limit criterion; it cannot be attributed solely to the contact predictor checking additional directions. Neither model supplies calibrated ground truth. Inputs initially beyond the limit are retained, but their large trajectory differences are not interpreted as ordinary stopping distances. A separate local-versus-Docker replay preserves all 144 actions and has observed maximum margin difference zero at the declared numerical tolerance; this is software reproducibility, not physical accuracy.

After inspecting these results, a 24-input subgrid at 15 and 20 mm, with speeds 0.08 and 0.24 m/s, is evaluated at physics steps of 1, 2, and 4 ms while retaining a 20 ms control interval. All 72 evaluations complete. The six initial disagreements persist, but one additional input changes its contact action with step size. At 15 mm, 0.24 m/s, zero age, and nominal friction, its forward contact margin is $-0.140$, $+0.035$, and $+0.422$ mm at the respective steps; the reduced margin remains $+0.600$ mm. This post hoc subgrid is not independent confirmation. Reaction-cutoff quantization also changes with the step, so the exercise is a sensitivity check rather than a convergence proof. A small positive contact margin is therefore insufficient evidence of numerical robustness in this fixture. No controller threshold or stopping parameter is fitted to these observations.

### 5.7 Pre-Hardware Software Rehearsal

A local UDP input gateway exercises continuous tag delivery through the existing mapping and runtime interfaces. The receiver supplies reception timestamps; senders cannot supply that field. It retains session and ordering checks and calls the runtime when no datagram arrives, allowing the existing silence watchdog to latch a hold. Commands are logged to a file rather than transmitted. Mapping uses the recorded-input clock convention, with synthetic and unverified mappings kept separate. This gateway is a local input-contract prototype, not an S7 driver, physical PLC connection, or demonstration of clock synchronization.

A six-item software inventory precedes eight combined check groups in the final Docker candidate. The recorded checks pass 126 core tests, eight web tests, three real loopback UDP fixtures, and the integration, reset, background, boundary, and qualification checks. The UDP fixtures cover silence followed by resumed data, repeated or wrong-session and malformed packets, and an unverified signal mapping. Returning data does not clear the latched hold. An earlier local run with a test fixture that reused equal decision times remains recorded as failed; the fixture is corrected before the complete Docker rerun. These checks support software consistency and inspectability, not measured process availability, physical fault-detection accuracy, or a hard real-time claim.

# 6 Learned Residual Prediction and Recent Qualification Studies

## 6.1 A Causal Supervised ETA Component

A first learned component now supplements the reduced model rather than replacing the controller. Let $t_a$ be the observed falling edge at the upstream optical sensor, $D_p$ the issued-command distance integrated over its preceding occupancy pulse, and $L$ the nominal workpiece length. The point response estimate is $\widehat{g} = L/D_p$. For the distance $d$ from that trailing-edge anchor to the expected downstream leading-edge event, the pulse-based prediction is

$$\widehat{t}_{\text{pulse}} = t_a + \frac{d}{u\widehat{g}}. \tag{9}$$

Only constant forward commands are admitted in this experiment. Pulse duration, issued speed, and their product form the three features $\phi$, frozen at $t_a$. The target is the subsequently observed downstream optical-edge time, not simulator position. Ridge regression learns the residual $r = t_{\text{edge}} - \widehat{t}_{\text{pulse}}$:

$$\min_{b,w} \sum_{i \in \mathcal{D}_{\text{tr}}} (r_i - b - w^\top \widetilde{\phi}_i)^2 + \lambda \|w\|_2^2. \tag{10}$$

Feature means and scales are obtained from training rows only; the intercept is not penalized. The corrected prediction is clipped below by the time when its features become available. This is supervised residual regression, not a trained fault classifier, policy learned by reinforcement learning, or demonstrated physical twin. It is not connected to actuator authority.

## 6.2 Separated Data and Prediction Results

The deterministic MuJoCo segment campaign contains 61 normal runs: 32 training, eight validation, eight error-envelope calibration, nine test, and four shifted-condition runs. Speed–gain–friction tuples do not overlap between splits. Training uses four commanded speeds, four gains, and two friction values. These are

**Table 6.** Sampled optical-edge arrival-time prediction: held-out normal test ($n = 9$) and shifted conditions ($n = 4$). MAE in ms; not physical timing accuracy.

| Method | Test MAE | Shift MAE |
|---|---|---|
| Nominal command kinematics | 105.7 | 330.0 |
| Upstream pulse kinematics | 98.5 | 83.3 |
| Pulse plus train-mean bias | 18.1 | 28.5 |
| Pulse plus Ridge residual | 15.4 | 38.3 |

Constant-bias reference added after the initial pilot; its value uses training rows only.

designed simulation conditions, not independently sampled factory observations. The admitted observation–command recording pairs are distinct only 16, four, four, nine, and four times in these respective splits: the friction variants in the first three splits produce identical records. No recording pair overlaps between splits. These duplicates are not independent repetitions. Candidate penalties 0.01, 0.1, 1, and 10 are selected by validation MAE; weights remain fitted on training rows. The model is sealed before calibration and test generation. A separate eight-run calibration set determines the maximum absolute corrected residual as an empirical error envelope. No statistical coverage guarantee follows from this grid.

The test Ridge MAE is 15.4 ms versus 98.5 ms for the pulse formula, but a train-mean residual correction already reaches 18.1 ms. The smaller 2.7 ms difference is the relevant incremental comparison; no significance claim is made. Moreover, shifted-condition MAE is 38.3 ms for Ridge versus 28.5 ms for the constant correction. The constant-bias reference was added after inspecting the initial pilot, is disclosed as post hoc, and does not use validation or test labels for its fitted offset. Ridge features and penalty selection were not revised from the test outcomes.

The empirical calibration radius is approximately 65.0 ms. It contains nine of nine test targets and four of four shifted targets, but the latter all fall outside the training feature box. These small denominators and the 20 ms optical sampling period preclude a physical sub-sample accuracy claim. Shifted predictions are diagnostic extrapolations, not authorized control inputs. An early data-generation attempt failed before a usable training run because nominal speed fell below an inherited degraded-speed setting; the isolated fixture setting was corrected locally without modifying the operational profile. The failed attempt is retained.

### 6.3 Closed-Loop Trade-Off and Release Qualification

A separate 144-run closed-loop segment comparison evaluates sequential, age-rule, and predictive speed selection across two provisional stopping windows, two speeds, two plant gains, three telemetry delays, and nominal or missing-target feedback. These windows are experimental criteria, not measured gripper tolerances. Under missing pickup feedback, predictive selection yields no upper-window exceedance in 24 runs, compared with 15 for the selected sequential

reference and 24 for the age-rule reference. This protection has a completion cost: in the tight healthy window, predictive selection achieves confirmed in-window stopping in only two of 12 runs, while the age-rule achieves 12. All three achieve 12 in the wider healthy window. No whole-line throughput gain is established. An initially incorrect adaptive timeout was corrected to integrate issued speed changes; its earlier pilot is excluded.

A further 72-run read-only qualification includes stalled-drive counterexamples. All 24 paired missing-sensor/stalled-drive cases have identical observation and logical decision traces, although final positions differ by 191–271 mm. No interval fits entirely within its allowed window after settling; no model-only release is issued. Zero false release claims therefore reflects abstention, not demonstrated diagnostic accuracy. An initial qualification bug incorrectly included speed immediately before the stop boundary; the corrected version uses issued distance and terminal command during the stopping interval. The original and corrected evidence are retained separately. Actual release remains blocked because model and actuation prerequisites have not been independently verified.

### 6.4 Advisory Event Monitoring with a Frozen Learned Residual

An advisory monitor now connects the frozen predictor to the existing upstream-pulse observer. For each of the pulse, train-mean correction, and Ridge alternatives, the event window is

$$I_m = [\max(t_a, \widehat{t}_m - r_m), \widehat{t}_m + r_m], \qquad r_m = \max_{i \in \mathcal{D}_{\text{cal}}} |t_{\text{edge},i} - \widehat{t}_{m,i}|. \tag{11}$$

The same eight normal calibration rows yield radii of 140.0, 42.9, and 65.0 ms, respectively; their recordings contain only four distinct pairs. A resolved target edge outside the window is flagged. In the absence of a seen target edge, two consecutive admissible source-time samples beyond the upper endpoint trigger a missing-edge finding. Receiver time does not substitute for the latest observed source time.

All three empirical alternatives share feature-box, quality, freshness, and history gates. An input outside the training feature box, a changed command, or a changed product length, sensor spacing, or sampling period selects the existing engineering pulse bounds instead. Lost edge history or unusable packets suppress model findings and retain a separate data-availability reason. These checks delimit use of this calibration; they do not validate every aspect of the model domain. The component consumes signal frames and locally issued commands. It is not connected to the central runtime policy or PLC output authority.

This exploratory study retrospectively reuses the nine normal test and four shifted recordings from Sect. 6.2 and adds 18 MuJoCo segments: missing-target feedback and a stalled drive for each of the nine normal conditions. Five scheduled delivery cases and three methods produce 465 replays from 31 recordings, not 465 independent production trials. Model weights and calibration radii remain fixed. Network cases are immediate reception, 40 ms delay, 120 ms stale

**Table 7.** Advisory ETA findings in fixed replay conditions; fault denominator combines nine missing-target and nine stalled-drive segments, which form nine identical input pairs.

| Method | Radius (ms) | Normal alerts no delay (/9) | Fault alerts no delay (/18) | Fault alerts 40 ms (/18) |
|---|---|---|---|---|
| Pulse | 140.0 | 0 | 18 | 16 |
| Constant bias | 42.9 | 0 | 18 | 18 |
| Ridge | 65.0 | 0 | 18 | 18 |

Two delayed pulse replays end before a late-edge confirmation window. Data rejection and engineering fallback are not credited as learned-model detections.

delivery, a brief post-pulse gap, and unavailable target-signal quality. These are shared-clock scheduling fixtures, not measurements of a physical network.

Under immediate delivery, each method flags all 18 fault segments and none of the nine normal segments. With 40 ms delay, Ridge and constant correction each flag 18 of 18; the uncorrected pulse alternative flags 16. In its two remaining replays the delivered recording ends before its late-edge confirmation window; these are censored observations, not established missed detections. Neither stale delivery nor the post-fit gap or quality-loss cases produces an authorized model finding; abstention is recorded separately. The four shifted normal records use engineering fallback under usable delivery.

Relative to constant correction, Ridge flags the immediate fault cases at the same time in two replays and 20–40 ms later in 16; it is never earlier. The 40 ms delivery case retains this pattern. Lower arrival-time MAE therefore does not imply faster anomaly detection when the calibrated window is wider. Moreover, all nine paired missing-sensor/stalled-drive inputs are identical, so these are findings of inconsistent expected progress, not uniquely isolated faults. These counts do not establish physical diagnostic accuracy, recovery, or improved production availability.

## 7 Physical Validation

*Reserved: no physical measurements or validation results are reported.*

### 7.1 Physical Setup and Calibration

### 7.2 Live PLC Synchronization and Network Measurements

### 7.3 Physical Fault Trials and Placement Accuracy

### 7.4 Comparison with Sequential PLC Operation

## 8 Discussion and Limitations

The evidence supports an inspectable software testbed, not a closed-loop physical twin. A supported hold is a virtual terminal condition, not demonstrated process availability or guaranteed safety. A speed cap can preserve completion in a delayed-data fixture without improving nominal cycle time. Selecting only completions would conceal interruption costs; the tables retain all planned runs.

The principal computation limitation is the gap between complete contact forecasts and the configured allowance. Optimization must preserve numerical behavior while being evaluated against actual elapsed time; increasing the allowance alone changes the question. Selected states and repeated host calls do not characterize worst-case scheduling. The engineering interval and stopping-coast assumptions require physical identification before quantified risk statements are justified.

Sparse binary sensing leaves sensor, communication, and model errors partly ambiguous. Findings indicate suspicions rather than uniquely isolated physical failure causes. CAD-to-simulation consistency, sensor offsets, belt slip, vacuum holding, pneumatics, and axis feedback remain unvalidated. Simultaneous products and continuing an interrupted old workpiece are outside the demonstrated capability. Section 7 is left blank rather than substituting synthetic measurements.

The learned component improves a sampled-event prediction metric in a small held-out grid; it does not resolve the observation ambiguity exposed by stalled-drive pairs. Its advantage over a train-mean correction is small and reverses in shifted conditions. The advisory extension now applies empirical windows and selected model-domain checks, but the observed wider Ridge window delays findings relative to a constant correction. Before inclusion in supervisory decisions, verified model-domain assumptions, decision budgets, and a matched

same-information controller comparison remain necessary. Learning and more conservative stopping are distinct from demonstrated reconfiguration that sustains production.

## 9 Conclusion

A modular network-aware supervisory framework has been implemented and examined virtually. The matched campaign demonstrates completion or supported interruption in its specified fixtures and makes information and computation boundaries explicit. The reduced predictor meets its allowance in the measured calls; the current independent MuJoCo predictor does not. Physical calibration, live integration, and a physical sequential-PLC comparison remain unreported. The later segment comparison and learned residual component do not establish complete-cycle resilience. These limitations delimit further work, rather than supporting claims of verified physical resilience or general performance superiority.

## Declarations Awaiting Author Completion

Authors and affiliations: _____

Funding and competing interests: _____

Author-approved AI-use disclosure: _____

## References


1. Ademujimi, T., Prabhu, V.: Digital twin for training Bayesian networks for fault diagnostics of manufacturing systems. Sensors **22**(4), 1430 (2022). https://doi.org/10.3390/s22041430
2. Cimino, C., Negri, E., Fumagalli, L.: Review of digital twin applications in manufacturing. Computers in Industry **113**, 103130 (2019). https://doi.org/10.1016/j.compind.2019.103130
3. Fu, B., Bi, M., Umeda, S., et al.: Digital twin-based smart manufacturing: Dynamic line reconfiguration for disturbance handling. IEEE Transactions on Automation Science and Engineering **22**, 14892–14905 (2025). https://doi.org/10.1109/TASE.2025.3563320
4. Fuller, A., Fan, Z., Day, C., Barlow, C.: Digital Twin: Enabling Technologies, Challenges and Open Research. IEEE Access **8**, 108952–108971 (2020). https://doi.org/10.1109/ACCESS.2020.2998358
5. Hespanha, J.P., Naghshtabrizi, P., Xu, Y.: A survey of recent results in networked control systems. Proceedings of the IEEE **95**(1), 138–162 (2007). https://doi.org/10.1109/JPROC.2006.887288
6. Jones, D., Snider, C., Nassehi, A., Yon, J., Hicks, B.: Characterising the Digital Twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology **29**, 36–52 (2020). https://doi.org/10.1016/j.cirpj.2020.02.002

7. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital Twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine **51**(11), 1016–1022 (2018). https://doi.org/10.1016/j.ifacol.2018.08.474
8. Negri, E., Fumagalli, L., Macchi, M.: A Review of the Roles of Digital Twin in CPS-based Production Systems. Procedia Manufacturing **11**, 939–948 (2017). https://doi.org/10.1016/j.promfg.2017.07.198
9. Rasheed, A., San, O., Kvamsdal, T.: Digital Twin: Values, Challenges and Enablers From a Modeling Perspective. IEEE Access **8**, 21980–22012 (2020). https://doi.org/10.1109/ACCESS.2020.2970143
10. Todorov, E., Erez, T., Tassa, Y.: MuJoCo: A physics engine for model-based control. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026–5033. IEEE (2012). https://doi.org/10.1109/IROS.2012.6386109
11. Uhlemann, T.H.-J., Lehmann, C., Steinhilper, R.: The Digital Twin: Realizing the Cyber-Physical Production System for Industry 4.0. Procedia CIRP **61**, 335–340 (2017). https://doi.org/10.1016/j.procir.2016.11.152
12. Yates, R.D., Sun, Y., Brown, D.R., III, Kaul, S.K., Modiano, E., Ulukus, S.: Age of information: An introduction and survey. IEEE Journal on Selected Areas in Communications **39**(5), 1183–1210 (2021). https://doi.org/10.1109/JSAC.2021.3065072